



به نام خدا

دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس امنیت سخت افزار

تمرین چهارم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	۸۱۰۱۰۳۰۹۸
تاریخ ارسال گزارش	۱۴۰۴/۰۴/۱
استاد	دکتر سیامک محمدی
ترم	بهار ۱۴۰۴

فهرست

سوال ۱: واترمارک با استفاده از استتیت های Don't Care	۴
صورت سوال:	۴
جدول صحت مدار (Truth Table):	۵
نحوه پیاده سازی واترمارک:	۷
ساده سازی تابع F_{new} :	۸
توضیح ساده سازی با جدول کارنو:	۹
دلیل تغییر نکردن عملکرد مدار:	۱۰
نحوه شناسایی واترمارک در صورت نیاز:	۱۱
سوال ۲: واترمارک با استفاده از استتیت های دونت کر انکدر	۱۲
صورت سوال:	۱۲
جدول صحت برای انکدر ۴ به ۲:	۱۳
نحوه پیاده سازی واترمارک:	۱۴
کد اسکی برای حروف رشته "SEC":	۱۴
روش پیاده سازی واترمارک در استتیت های دونت کر مدار:	۱۴
جدول کارنو چهار متغیره برای انکدر ۴ به ۲:	۱۶
حالت بدون واترمارک:	۱۶
حالت با واترمارک:	۱۷
سوال ۳: واترمارک با استفاده از گراف	۲۰
صورت سوال:	۲۰
روش حل:	۲۰
توضیحات و روند مرحله به مرحله:	۲۱

شکل‌ها

- شکل ۱- تعریف و یه مثال از استتیت های Don't Care..... ۴
- شکل ۲- مثال جزوه از نحوه امبد کردن واترمارک در استتیت های Don't Care..... ۵
- شکل ۳- تعریف مینترم ها ۵
- شکل ۴- جدول صحت سوال ۱ ۶
- شکل ۵- مقداردهی به استتیت های Don't Care برای پیاده سازی واترمارک..... ۷
- شکل ۶- شکل جدول کارنو چهارمتغیره و نحوه قرار گرفتن مینترم ها در آن..... ۸
- شکل ۷- تابع ساده شده با جدول کارنو ۸
- شکل ۸- نحوه انجام واترمارک و بازیابی آن..... ۱۱
- شکل ۹- تصویری از انکدر ۴ به ۲ ۱۲
- شکل ۱۰- جدول صحت برای انکدر ۴ به ۲..... ۱۳
- شکل ۱۱- نحوه جاسازی واترمارک در حالت‌های دونت کر ۱۵
- شکل ۱۲- مثالی از جدول کارنو ۴ متغیره برای انکدر ۴ به ۲ ۱۶
- شکل ۱۳- محاسبه خروجی X با استفاده از جدول کارنو برای انکدر ۴ به ۲ ۱۶
- شکل ۱۴- محاسبه خروجی Y برای انکدر ۴ به ۲ ۱۷
- شکل ۱۵- استتیت های معتبر دیزاین پس از اضافه کردن ماترمارک ۱۷
- شکل ۱۶- دو جدول کارنو برای محاسبه فرم SOP خروجی های انکدر پس از واترمارکینگ..... ۱۸
- شکل ۱۷- روند آپدیت گراف و پیاده سازی واترمارک..... ۲۱

جدولها

جدول ۱- واتر مارک ۱۰۱.....۱۰

سوال ۱: واترمارک با استفاده از استیت های Don't Care

صورت سوال:

Hardware Security & Trust — Homework 4

Question 1 – Watermarking Using Don't-Care Conditions

Given

$$F(a, b, c, d) = \Sigma m(1, 3, 4, 6, 9, 11), D = \Sigma d(0, 5, 15)$$

Tasks

1. Embed the 3-bit author signature “101” in F using the don't-care watermarking technique.
 - Update the truth-table entries for minterms 0, 5 and 15.
2. List the ON-minterms of the water-marked function F_{wm} .
3. Minimise $F_{wm}(a, b, c, d)$ with Karnaugh maps and give the simplified SOP form.
4. Explain briefly
 - why normal circuit operation is unchanged, and
 - how the 3-bit signature can later be detected and verified.

خب در این سوال همان طور که در صورت سوال نیز ذکر شده است باید با استفاده از استیت های Don't Care در تابع فانکشن دیزاینی که داده شده است بیایم و واترمارک ۱۰۱ رو به اصطلاح در دیزاین Embed کنیم.

خب همان طور که از درس مدار منطقی به یاد داریم استیت های Don't Care می‌توانستند مقدار صفر یا یک به خود بگیرند و در کارکرد مدار نیز تاثیری نداشتند و جزو فانکشن اصلی مدار به حساب نمی‌ومدن.

◆ AI Overview

In digital logic design, don't care conditions (also known as don't care terms) are **input combinations where the output of a circuit doesn't matter, or is unspecified**, and can be treated as either a 0 or a 1 during simplification of the logic expression. These conditions arise when certain input combinations are impossible or when the output doesn't affect the circuit's functionality. 

AB \ CD	00	01	11	10
00	X	1	X	1
01		X	1	1
11	X	1	1	1
10	1			

شکل ۱- تعریف و یه مثال از استیت های Don't Care

حال ما برای امبد کردن واترمارک ۱۰۱ از روش مشابه مثال جزوه استفاده میکنیم.

Example of Watermarking Using Don't-Care Conditions (1)

Step 2: Embed a Watermark by Assigning Values

- We **force** the output of the don't-care terms to specific values to encode a **binary signature**.
- Example:
 - To embed "01", we set $a'b'c'd' = 0$ and $abcd = 1$.
 - To embed "10", we set $a'b'c'd' = 1$ and $abcd = 0$.

شکل ۲- مثال جزوه از نحوه امبد کردن واترمارک در استیت های Don't Care

خب همان طور که در تصویر بالا مشاهده میکنید برای امبد کردن واترمارک مثلا ۰۱ میایم و مقدار اولین استیت Don't Care رو برابر با صفر و دومین استیت Don't Care رو برابر با یک قرار میدهیم. و سپس باید عبارت هایی که برای استیت های دونت کر با مقدار یک هستند رو به تابع مدار اضافه کنیم و آن را ساده کنیم تا به جواب نهایی برسیم.

جدول صحت مدار (Truth Table):

خب با توجه به اطلاعاتی که در صورت سوال داده شده به راحتی میتونیم جدول صحت مدار رو رسم کنیم و استیت های ۱ (مینترم ها) ، استیت های صفر مدار و همچنین استیت های دونت کر رو مشخص کنیم.

AI Overview

In Boolean algebra, a minterm is a **product (AND) term where each variable in the expression appears exactly once, either in its true form or complemented form**. Each minterm corresponds to a unique combination of input variables that results in a true (1) output. Minterms are fundamental in representing Boolean functions in a Sum-of-Products (SOP) form. 

x	y	z	F	Minterms
0	0	0	0	m_0
0	0	1	1	m_1
0	1	0	0	m_2
0	1	1	0	m_3
1	0	0	1	m_4
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7

شکل ۳- تعریف مینترم ها

خب از جایی که در صورت سوال خواسته شده تا از فرم SOP برای نوشتن تابع نهایی استفاده کنیم

با مینترم ها سر و کار داریم. خب مینترم همان طور که در صورت سوال نیز ذکر شده و در تصویر صفحه قبل هم قابل مشاهده است به AND یکسری متغیره گفته میشود که در این عبارت هر متغیر فقط یکبار ظاهر میشود.

جدول صحت سوال ۱

$$F(a,b,c,d) = \sum_{\text{minterms}} (1, 3, 4, 7, 9, 11), D = \sum d(0, 5, 15)$$

Don't care

میت های ۰، ۵ و ۱۵ طبق گفته سوال Don't care هستند.

در جدول رو برو ۱۶ مینترم داریم که ۶ از آن ها مقدار یک دارند که این مقدار یک ها می توانند تابع مادر خروجی خود مقدار ۱ به خود می گیرند.

جدول کارنو برای این تابع :

cd \ ab	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

گزارش نحوه قرار گرفتن مینترم ها در جدول کارنو //

cd \ ab	00	01	11	10
00	X	1	1	0
01	1	X	0	1
11	0	0	X	0
10	0	1	1	0

خب برای Embed کردن ماتریک ۱۵ ای می آیم و مقدار مینترم های ۰، ۵ و ۱۵ رو برابریت و m_5 رو برابر صفر قرار می دهیم.

شکل ۴- جدول صحت سوال ۱

خب همان طور که در تصویر بالا مشاهده میکنید جدول درستی مربوط به این تابع و جدول کارنو مربوط به آن رسم شده اند.

خب طبق داده های اولیه ای که در سوال به ما داده شده مینترم های مربوط به تابع اصلی به شرح زیر میباشند.

$$F(a, b, c, d) = \sum m(1, 3, 4, 6, 9, 11)$$

همچنین سه تا مینترم هم مربوط به استیت های دونت کر میباشد.

$$D = \sum d(0, 5, 15)$$

نحوه پیاده سازی واترمارک:

خب حال برای امضای دیجیتال یا همان واترمارک خود به نحوه زیر عمل میکنیم.

$$F = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd$$

عبارت بالا تابع F را نشان می دهد حال عبارت های $a'b'c'd'$ و $abcd$ را به آن اضافه کنیم به عنوان واترمارک

$$D = \sum d(0, 5, 15)$$

$$m_0 = a'b'c'd' = 1$$

$$m_5 = a'bc'd = 0 \rightarrow \text{اضافه نمی شود.}$$

$$m_{15} = abcd = 1$$

$$F_{new} = F + abcd + a'b'c'd'$$

$$F_{new} = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd + a'b'c'd' + abcd$$

شکل ۵- مقداردهی به استیت های Don't Care برای پیاده سازی واترمارک

خب همان طور که مشخص است به مینترم های Don't Care ، m_0, m_{15} مقدار یک و به مینترم Don't Care ، m_5 مقدار صفر میدهیم.

عبارت های مربوط به مینترم های Don't Care که مقدار یک گرفته اند رو به تابع اصلی اضافه میکنیم و F_{new} بدست می آید.

خب در مرحله بعدی باید با استفاده از جدول کارنو ۴ متغیره تابع داده شده را ساده کنیم تا به فرم ساده شده SOP مد نظر سوال برسیم.

جدول کارنو چهار متغیره و نحوه قرار گرفتن مینترم ها در آن را میتونید در شکل زیر مشاهده کنید.

AB \ CD				
	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

شکل ۶- شکل جدول کارنو چهارمتغیره و نحوه قرار گرفتن مینترم ها در آن

خب حال به ساده کردن تابع میپردازیم:

ساده سازی تابع F_{new} :

۱۲	۱	۱	۰	۰	۰
۱۳	۱	۱	۰	۱	۰
۱۴	۱	۱	۱	۰	۰
۱۵	۱	۱	۱	۱	X

ab \ cd	00	01	11	10
00	0	0	1	0
01	1	0	0	1
11	0	0	1	0
10	0	1	1	0

خب برای Embed کردن ماتریک ۱۵ می‌ایم و مقدار مینترم های m_0 و m_{15} رو برابر ۰ و m_5 رو برابر ۱ می‌ذاریم.

ab \ cd	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	0	0	1	0
10	0	1	1	0

$$F = \overset{(1)}{a'b'c'} + \overset{(2)}{a'b'd} + \overset{(3)}{acd} + \overset{(4)}{b'd}$$

تاجای مطلق از دسته های بنبر لته
 ۱- استفاده از دسته اول هم لته
 ۲- یه های تنها برای دسته بندی شروع داریم.

شکل ۷- تابع ساده شده با جدول کارنو

$$F = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd$$

نکته: عبارت بالا تابع F را نشان می‌دهد حال عبارت‌های $a'b'c'd'$ و $abcd$ را به آن اضافه کنیم به عنوان مانتراکت

$$D = \sum d(0, 5, 15)$$

$$m_0 = a'b'c'd' = 1$$

$$m_5 = a'b'cd = 0 \rightarrow \text{اضافه نمی‌شود}$$

$$m_{15} = abcd = 1$$

$$F_{\text{new}} = F + abcd + a'b'c'd'$$

$$F_{\text{new}} = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd + a'b'c'd' + abcd$$

simplify F_{new} (حذف جملات)

$$F_{\text{new}} = a'b'c' + a'bd' + acd + b'd$$

توضیح ساده سازی با جدول کارنو:

همان طور که در جدول کارنو هم مشاهده میکنید اومدیم و برای اولین دسته یک ها مینترم ۰ و ۱ رو با هم گرفتیم (ساده کردیم) و برای دسته دوم مینترم ۴ و ۶ رو با هم ساده کردیم. برای دسته سوم مینترم های ۱۱ و ۱۵ رو ساده کردیم و برای آخرین دسته هم مینترم های ۳ و ۱۱ و ۹ رو با هم در نظر گرفتیم و ساده اش کردیم.

گرفتن دسته ها در جدول کارنو دو تا قانون کلی داشت تا این که ساده ترین فرم تابع بدست بیاد:

۱. اول از یک های تنها شروع میکنیم که یکی در نزدیکی آن ها نیست

۲. بزرگترین دسته ممکن که توان ۲ باشد رو در نظر میگیریم.

برای نوشتن عبارت SOP هم چون جدول کارنو برای مینترم ها بوده کار سختی نداریم و به راحتی کافیه عبارت‌ها رو برای هر دسته یک هایی که گرفتیم بنویسیم و آنها را با هم جمع کنیم (فرم SOP).

نکته) الان سطر اول جدول کارنو مقدار $a = 0, b = 0$ میباشد به همین شکل برای سطر دوم

$a = 0, b = 1$ میباشد و غیره ، برای ستون ها هم همین روند صادق است با این تفاوت که متغیرها c, d هستند.

برای بخش آخر به طور خلاصه به روند کاری که انجام دادیم میپردازیم.

در صفحه بعد میتونید این مورد رو مشاهده کنید.

Step 1: Start with a Boolean Function:

$$F(a, b, c, d) = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd$$

This function has three don't care conditions according to question:

Step 2: Embed a Watermark by assigning values:

To embed "101" We set $a'b'c'd' = 1$ and $a'bc'd = 0$ and $abcd = 1$

Step 3: Simplify the function with watermark

1. Add new constraints:

$$F_{new} = a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + ab'c'd + ab'cd + a'b'c'd' + abcd$$

2. Simplify:

$$F_{new} = a'b'c' + a'bd' + acd + b'd$$

Result: Watermark "101" is embedded without affecting normal functionality

Final Conclusion Table:

جدول ۱- واتر مارک ۱۰۱

Watermark	Forced Don't Cares	Simplified Function
101	$abcd = 1, a'b'c'd' = 1$	$a'b'c' + a'bd' + acd + b'd$

نکته خیلی مهم) با توجه به اینکه در ساده سازی ما میتونستیم بیایم و مینترم ۰ رو هم با ۱ و هم با ۴ در نظر بگیریم پس این تابع دو تا فرم SOP ساده شده دارد و unique نیست.

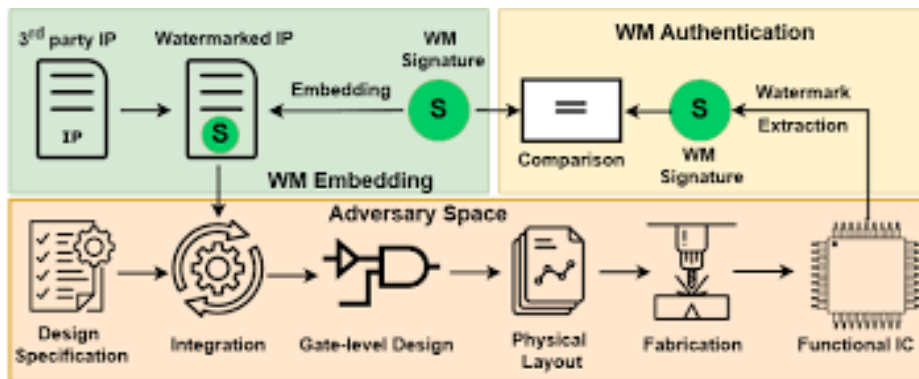
دلیل تغییر نکردن عملکرد مدار:

همان طور که در ابتدا هم بیان کردیم برای پیاده سازی واترمارک خودمون ما فقط از استیت های Don't Care مدار استفاده کردیم و همان طور که میدانیم این استیت ها در عملکرد مدار تغییری ایجاد نمیکنند و در این استیت ها خروجی مدار رو نگاه نمیکنیم و برامون مهم نیست، پس مقدار دهی چه صفر و چه یک به این استیت ها هیچ تغییری در عملکرد مدار ایجاد نخواهد کرد.

نکته اضافی: از درس مدار منطقی یادمون هست که کمک این استیت ها و جدول کارنو میتونستیم هم فرم تابع رو ساده تر بنویسیم (هر جا که یک دادن به این استیت های دونت کر باعث بزرگتر شدن دسته های یک میشد اینکار رو میکردیم) و همچنین جلوی بروز هازاد رو هم میتونستیم بگیریم که این مورد به خاطر مجاورت ۱ ها و استیت های دونت کری بود که در یک دسته گرفته نشده بودند.

نحوه شناسایی واترمارک در صورت نیاز:

خب ما از مطالب درس به یاد داریم که دلیل پیاده سازی واترمارک در دیزاین این بود که بعداً اگر توسط کسی به سرقت رفت و یا جعل شد بتوانیم بعداً در دادگاه مالکیت اون چیپ رو اثبات کنیم، واترمارک نباید کارکرد مدار رو عوض میکرد و همچنین نباید اورهد سخت افزاری زیادی میداشت که یکی از روش های خوب برای پیاده سازی ساده واترمارک همین روش Don't Care States بود. زمانی که بخواهیم در دادگاه ثابت کنیم که این واترمارک متعلق به ماست (اثبات مالکیت) تنها کافیست خروجی های ۳ استیت دونت کر مدار رو بخونیم و اگر مقدار "۱۰۱" داشته باشند یعنی واترمارک معتبره و متعلق به ماست. منظور از ۳ استیت دونت کر همان مینترم های ۰،۵،۱۵ بود که برای پیاده سازی واترمارک از آنها استفاده کردیم و مقدار مینترم ۰،۱۵ رو برابر با ۱ و مینترم ۵ رو برابر با ۰ گرفتیم تا واترمارک ۱۰۱ رو پیاده سازی کنیم. خب پس بعداً هم همان طور که گفته شد با خواندن خروجی های این مینترم ها (استیت های دونت کر) میتونیم واترمارک ۱۰۱ رو بازیابی کنیم.



شکل ۸- نحوه انجام واترمارک و بازیابی آن

سوال ۲: واترمارک با استفاده از استیت های دونت کر انکدر

صورت سوال:

Question 2 – Watermarking “SEC” in a 4-to-2 Encoder

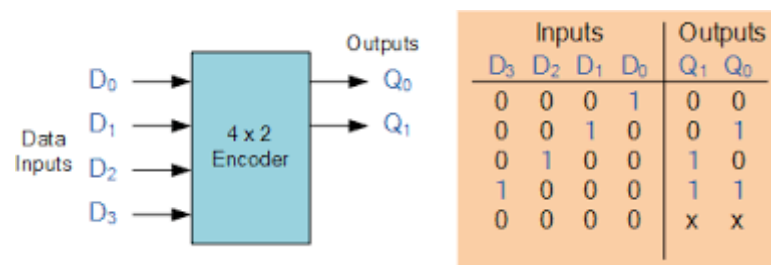
A 4-to-2 encoder is valid only when **exactly one** of the inputs a,b,c,d is high; the other 12 patterns are don't-cares.

Embed the 3-letter string “SEC” (8-bit ASCII per letter) into these don't-care patterns using the class method:

1. Write the 24-bit ASCII stream for “SEC”.
2. Starting **from the right-hand end**, split the bit stream into 5-bit groups.
*If the final group is incomplete, append zero(s) to the **right** until the length is a multiple of 5.*
3. Record the new input–output pairs and present the updated truth table.
4. Using 2-variable Karnaugh maps for XX and YY, give the minimal additional product terms and justify that normal encoder operation is unaffected.

در این سوال به ما یک انکدر ۴ به ۲ داده شده است و از ما خواسته شده است تا طبق روشی که در اسلایدهای درس برای استفاده از استیت های دونت کر انکدر ۴ به ۲ معرفی شده است واترمارک خواسته شده را پیاده سازی کنیم.

خب در ابتدا بیایید تعریف ساده ای از انکدر داشته باشیم:



شکل ۹- تصویری از انکدر ۴ به ۲

خب همان طور که در تصویر مشاهده میکنید و در صورت سوال هم توضیح داده شده است این انکدر تنها زمانی خروجی معتبر دارد که فقط یکی از چهار ورودی a, b, c, d آن مقدار یک داشته باشند و بقیه صفر باشند پس در مجموع ۴ ترکیب از ۱۶ ترکیب ممکن ورودی ها معتبر است و بقیه نامعتبر است (همان استیت های Don't Care) که از این ۱۲ ترکیب باید استفاده کنیم تا واترمارک خودمون رو توی دیزاین امبد کنیم.

مثل مثال قبل در اینجا هم ابتدا جدول درستی رو رسم کرده و سپس روش پیاده سازی واترمارک رو قدم به قدم توضیح خواهیم داد.

جدول صحت برای انکدر ۴ به ۲:

Truth Table Encoder 4 to 2 :

جدول صفت سوال (۲)

چهار ترکیب مقبضه برای خروجی سطرهای ۱، ۲، ۳، ۴، ۵ هستند

در این حالت هاستهای زیر چهار ویدیو (a, b, c, d) مقدار 1 دارند.

۱۲. تربیه (سطر) Don't care داریم نه از این صاکت برای چاره سازی
ماترمارک است (ده می نشی).

	a	b	c	d	output
0.	0	0	0	0	XX
1.	0	0	0	1	00
2.	0	0	1	0	01
3.	0	0	1	1	XX
4.	0	1	0	0	10
5.	0	1	0	1	XX
6.	0	1	1	0	XX
7.	0	1	1	1	XX
8.	1	0	0	0	11
9.	1	0	0	1	XX
10.	1	0	1	0	XX
11.	1	0	1	1	XX
12.	1	1	0	0	XX
13.	1	1	0	1	XX
14.	1	1	1	0	XX
15.	1	1	1	1	XX

INPUT	OUTPUT
1000	00
0100	01
0010	10
0001	11

Don't care states:

INPUT	OUTPUT
0000	XX
0011	XX
0101	XX
0110	XX
0111	XX
1001	XX
1010	XX
1011	XX
1100	XX
1101	XX
1110	XX
1111	XX

step 1) 24 bit ASCII for 'Sec'

: Bitstream

$S \rightarrow 01010011$

$E \rightarrow 01000101$

C → 0100011

[illegible]

↑
تات طبتی لفته سوال یک
صفحه سمت راست اضافه می کنیم
تا ۵ تا ۶

حال به دست‌های پنج‌نای تقسیم می‌کنیم

صفر اعشاری سده به انتهای رسته

از متراست به چپ شروع می کنیم .

شکل ۱۰- جدول صحت برای انکدر ۴ به ۲

همان طور که پیش تر هم گفته شد جدول صحت دارای ۱۶ سطر است که ۴ تای آن معتبر و ۱۲ تای آنها حالت‌های نامعتبر است.

نحوه پیاده سازی واترمارک:

همان طور که در اولین مرحله صورت سوال هم بیان شده باید کد اسکی برای هر از حروف SEC رو پیدا کنیم:

کد اسکی برای حروف رشته "SEC":

کد اسکی برای حرف S:

$S \rightarrow 01010011$

کد اسکی برای حرف E:

$E \rightarrow 01000101$

کد اسکی برای حرف C:

$C \rightarrow 01000011$

با چسبوندن این بیت ها به هم به یک رشته بیتی ۲۴ بیتی میرسیم:

Bitstream: 010100110100010101000011

حال باید از سمت راست پنج تا پنج تا جدا کنیم تا به پایان رشته برسیم. از جایی که رشته ما ۲۴ بیتی بود گروه آخر یک بیت کم میاره که برای حل این مشکل طبق گفته صورت سوال یک بیت صفر به سمت راست اضافه میکنیم تا به ۵ تا گروه ۵ بیتی برسیم.

نکته مهم) از جایی که در صورت سوال این مورد به طور واضح روشن نشده است ما دو روش رفتیم روش اول یک بیت صفر به انتهای رشته ۲۴ بیتی مون از سمت راست اضافه کردیم و در روش دوم یک بیت صفر به انتهای سمت راست گروه پنجم (که ۴ بیت بود) اضافه کردیم. اما حل کامل مون برای فرض اول است.

روش پیاده سازی واترمارک در استتیت های دونت کر مدار:

در مرحله دوم پس از شکل دادن بیت استریم باید حالا استتیت های دونت کری که میخوایم واترمارکمون رو در اون تعبیه کنیم رو انتخاب کنیم برای اینکار ۳ بیت از سمت راست رشته بیتی جدا میکنیم حالا چرا ۳ تا؟ چون ۱۲ تا استتیت Don't Care داریم که برای آدرس دهی و انتخاب از بین شون به $\log_2 12 = 3\text{bit}$ احتیاج داریم و بعد از این دو بیت بعدی از بیت استریم رو در اون جایگاه دونت کر جاسازی میکنیم (قرار میدهیم). دلیل اینکه به دسته های ۵ بیتی در ابتدا بیت استریم رو تقسیم کردیم هم همین بود که ۳ بیت برای آدرس Don't Care مربوطه و ۲ بیت هم مربوط به محتوایی که میخوایم جای اون استتیت دونت کر بذاریم در نظر گرفته شده است.

در تصویر زیر روال کار مشخص است:

شکل ۱۱- نحوه جاسازی واترمارک در حالت‌های دونت کر

خب در ادامه باید این حالت‌های دونت کر که مقدار جدید گرفتند رو به ۴ حالت معتبر قبلی اضافه کنیم و سپس دوباره تابع رو برای مدار بنویسیم.

جدول کارنو چهار متغیره برای انکدر ۴ به ۲:

خب همان طور که مشاهده میکنید چون خروجی ما ۲ بیتی است به دو جدول کارنو نیاز داریم.

$D_3 D_2$ \ $D_1 D_0$		D_1				
		00	01	11	10	
00	X					D_2
01	1	1	1	1		
11	1	1	1	1		
10	1	1	1	1		
		D_0				
		$A_1 = D_2 + D_3$				

$D_3 D_2$ \ $D_1 D_0$		D_1				
		00	01	11	10	
00	X			1	1	D_2
01						
11	1	1	1	1		
10	1	1	1	1		
		D_0				
		$A_0 = D_3 + D_1 \bar{D}_2$				

شکل ۱۲- مثالی از جدول کارنو ۴ متغیره برای انکدر ۴ به ۲

در نمونه سوال این طور فرض شده که **بیت اول خروجی مون از سمت چپ X** و **بیت دوم خروجی مون Y** نام دارد و برای هر کدام یک جدول کارنو رسم کرده و با دسته بندی یک ها تابع رو برای حالت عادی و بدون واترمارک انکدر ۴ به ۲ نوشته است در زیر تصاویر این مورد موجود است:

حالت بدون واترمارک:

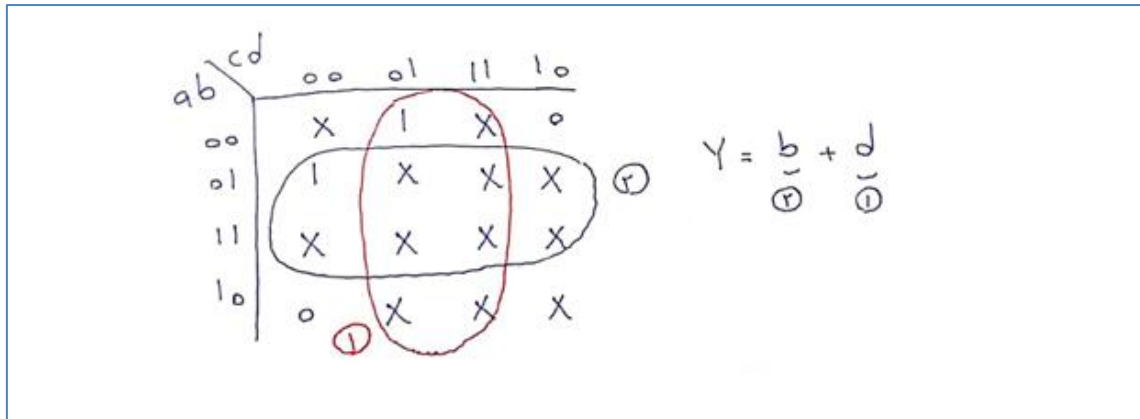
(Before watermark)

ab \ cd		00	01	11	10
		00	X	1	1
01	0		X	X	X
11	X		X	X	X
10	0		X	X	X

$X = \frac{c}{0} + \frac{d}{0}$

① ②

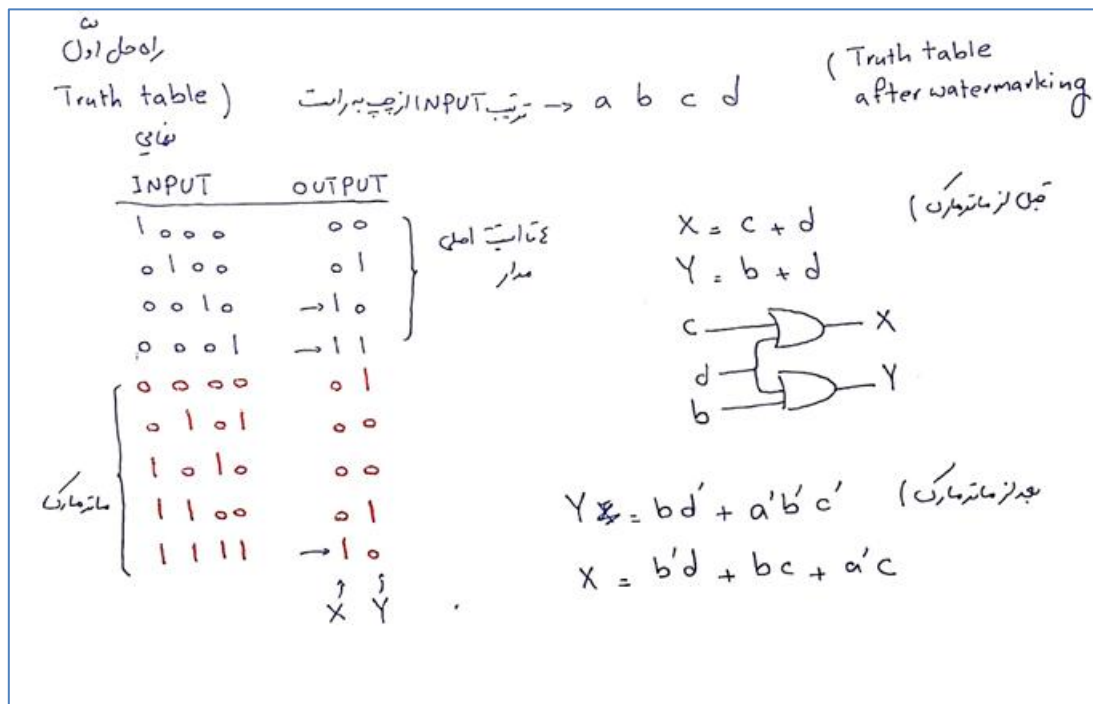
شکل ۱۳- محاسبه خروجی X با استفاده از جدول کارنو برای انکدر ۴ به ۲



شکل ۱۴- محاسبه خروجی Y برای انکدر ۴ به ۲

حالت با واترمارک:

اول باید همان طور که پیشتر هم گفته شد بیایم و استیت های دونت کری که مقدارشون تغییر کرد در فرآیند جاسازی واترمارک رو به حالت های معتبر اضافه کنیم و سپس دوباره با جدول کارنو تابع ساده شده به فرم SOP رو برای هر دو خروجی انکدر بنویسیم.



شکل ۱۵- استیت های معتبر دیزاین پس از اضافه کردن ماترمارک

در صفحه بعد محاسبات و جدول کارنو و دسته بندی یک ها برای نوشتن فرم SOP کمینه رو هم میتونید مشاهده کنید.

جدول کارنو برای X (بیت اول خروجی)

	cd	00	01	11	10
ab	00	0	1	X	1
	01	0	0	X	X
	11	0	X	1	X
	10	0	X	X	0

ضرب حال به X ها مقادیر 0 یا 1 می توانیم بهریم جوری باید مقدار بهریم که 1 های نه داریم پوشش داده بشن توسط بزرگترین دسته 1 مسن می در همین حال ساده ترین حالت SOP می هم بدست بیاد و عبارت اضافه نداشته باشه.

$$X = \underbrace{b'd}_{(1)} + \underbrace{bc}_{(2)} + \underbrace{a'c}_{(3)} \leftarrow \text{after watermarking}$$

جدول کارنو برای Y (بیت دوم خروجی)

	cd	00	01	11	10
ab	00	1	1	X	0
	01	1	0	X	X
	11	1	X	0	X
	10	0	X	X	0

$$Y = \underbrace{bd'}_{(1)} + \underbrace{a'b'c'}_{(2)}$$

نکته! در اینجا می توانیم میتریم m_1 رو دسته چهارتا هم بگیریم (با m_3, m_9, m_{11}) ولی باز هم m_0 پوشش داده نمی شه و عبارت طولانی می شه پس اومدیم و با بقیه دنیجی در نظر گرفتیم.

شکل ۱۶- دو جدول کارنو برای محاسبه فرم SOP خروجی های انکدر پس از واترمارکینگ

دقت کنید نکته ای که برای گرفتن دسته های یک و ساده سازی برای رسیدن به فرم sop کمینه وجود دارد این است که ما مقادیر X یا همون Don't Care ها مون رو میتونیم صفر یا یک بگیریم حالا میایم و جوری مقدار دهی میکنیم که بشه دسته های بزرگتری از یک ها رو با هم گرفت که در نتیجه خروجی به ساده ترین فرم خودشون تبدیل شود.

پس در نتیجه مقدار خروجی X برای حالت واترمارک شده:

$$X = b'd + bc + a'c$$

و مقدار خروجی Y برای حالت واترمارک شده:

$$Y = bd' + a'b'c'$$

در صفحه بعدی ابتدای حل با فرض دوم هم قرار داده شده است.

(راه حل دوم)

صفر رو به سمت راست آخری
گروه (نمره ۵) اضافه کردیم.

INPUT	OUTPUT
۰- ۰۰۰۰	XX
۱- ۰۰۱۱	XX
۲- ۰۱۰۱	XX
۳- ۰۱۱۰ ← ۰۰	① ← X
۴- ۰۱۱۱ ← ۰۰	④
۵- ۱۰۰۱	XX
۶- ۱۰۱۰ ← ۰۱	②
۷- ۱۰۱۱ ← ۰۱	⑤
۸- ۱۱۰۰ ← ۱۰	③
۹- ۱۱۰۱	XX
۱۰- ۱۱۱۰	XX
۱۱- ۱۱۱۱	XX

دقت شود که در این حالت فرض کردیم که صفر رو به سمت راست آخرین گروه اضافه کردیم که در تصویر بالا نیز مشخص است و روال حل مثل قبل است.

نکته مهم) در طول جاسازی واترمارک چون آدرس ها نسبی بود یک جایگاه تکراری در اومد که اومدیم و در اولین جایگاه دونت کری که مقدار نگرفته بود مقدار رو جاسازی کردیم.

در انتها به این سوال پاسخ میدهم که آیا این واترمارک تاثیری در عملکرد مدار میگذارد؟

خب همان طور که در سوال قبل هم توضیح دادیم چون برای پیاده سازی این واترمارک از استیت های Don't Care فقط استفاده کردیم و در حالت عادی هم مقدار خروجی در این استیت ها برای ما مهم نیست پس طبیعتاً این واترمارک تاثیری در عملکرد عادی مدار نخواهد گذاشت چون کلا ۴ تا استیت معتبر داریم و در حالت های دیگر اصلاً خروجی مدار برامون مهم نیست ولی خب با خوندن همین خروجی استیت ها در حالت های Don't Care میتونیم به طور مشابه سوال قبل واترمارک رو بازیابی کنیم.

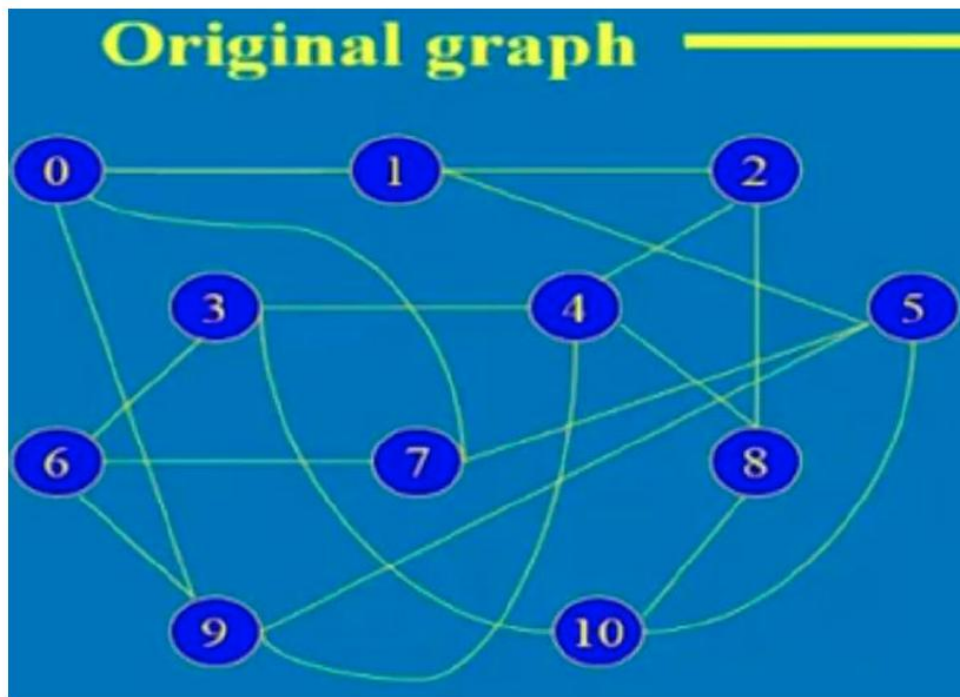
سوال ۳: واترمارک با استفاده از گراف

صورت سوال:

Question 3 – Watermarking a Graph-Coloring Solution

Using the edge-insertion rule shown in the lecture slides, embed the **12-bit** binary string to below graph.

101001011111101001011111



روش حل:

در ابتدا رشته ۲۴ بیتی واترمارک رو داریم که در صورت سوال داده شده است:

Bitstring: 101001011111101001011111

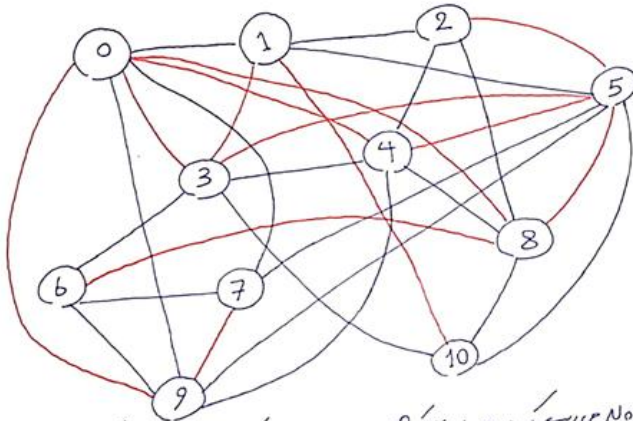
حال از سمت چپ به راست شروع میکنیم به حرکت کردن و از نود 0 تا 10 رو به ترتیب پیمایش میکنیم. برای هر نود دو همسایه نزدیک از شماره های بعدی که به این نود یال ندارند رو مشخص میکنیم و اگر بیت واترمارک صفر بود به نود همسایه با شماره نزدیک تر و اگر یک بود به همسایه دورتر وصل میکنیم همین روند رو ادامه میدهیم تا به نود ۱۰ برسیم و بعد از آن دوباره از نود ۰ شروع میکنیم. رشته بیتی داده شده در صورت سوال ۲۴ بیت است ولی در صورت سوال خواسته که ۱۲ بیت رو در گراف پیاده سازی کنیم و برای همین هم ۱۲ بیت سمت اول (سمت چپ) رشته بیتی واترمارک رو پیاده سازی کردیم ولی روال برای ۲۴ بیت هم به همین صورت است و ادامه دارد.

توضیحات و روند مرحله به مرحله:

کلمه (طبق خواسته سوال و با توجه به اینکه شکل شلخ نزدیکی ۱۲ بیت است چه رشته پیاده سازی انجام داریم):

24 bit binary string:

→ 1 0 1 0 0 1 1 1 1 1 0 1 0 0 1 0 1 1
Node 1 2 3 4 5 6 7 8 9 10 ... 11 1



از سمت چپ شروع می کنیم ابتدا برای Node 0 دوهایه نزدیک به
به یال ندارند را پیدا می کنیم و اگر بیت واترمارک من برابر صفر بود به
نود با شماره نزدیک تر واترمارک بود به نود با شماره دورتر یال دارند
قرار وصل می کنیم.

اگر نوبت نزدیک به یال ندارند
نود ۳، ۲ هستند که چون بیت واترمارک من برابر
۱ است به نود با شماره دورتر وصل می کنیم.

در وسط بعد به نود شماره Node 1 می رسیم که دوهایه
نزدیکی که بهم یال ندارند نود ۳، ۴ می باشد.
که چون بیت واترمارک صفر است به نود ۳ یال می کنیم.
در وسط بعدی به نود 2 می رسیم که دوهایه
نزدیکی نودهای ۳، ۴ هستند که چون بیت واترمارک برابر
یک است به هایه دورتر یال رسم می کنیم

در ادامه به Node 3 می رسیم که دوهایه نزدیکی ۵ و ۷ هستند که چون بیت واترمارک صفر است به ۵ یال وصل
می کنیم. در ادامه به Node 4 می رسیم که دوهایه نزدیکی ۵ و ۶ هستند و چون بیت واترمارک صفر است به ۵ یال
می کنیم. در ادامه به Node 5 می رسیم که دوهایه نزدیکی ۶ و ۸ هستند که چون بیت واترمارک یک است به ۸ یال
وصل می کنیم. برای Node 6 هایه ۸ و ۱۰ هستند که چون بیت واترمارک صفر است به ۸ وصل می کنیم.
برای Node 7 دوهایه نودهای ۸ و ۹ هستند که چون مقدار بیت واترمارک ۱ است به ۹ یال وصل می کنیم.
برای Node 8 دوهایه نزدیک نودهای ۹ و ۱۰ هستند که چون بیت واترمارک ۱ است به ۱۰ یال وصل می کنیم.
برای Node 9 دوهایه نزدیک ۱۰ و ۵ هستند که چون بیت واترمارک ۱ است به صفر وصل می کنیم.
برای Node 10 دوهایه نزدیک ۵ و ۸ هستند که چون واترمارک ۱ است به ۱۰ یال وصل می کنیم و

شکل ۱۷- روند آپدیت گراف و پیاده سازی واترمارک

روند اضافه کردن یال ها طبق توضیحات صفحه قبل و اسلایدهای درس میباشد و توضیحات کامل
و مرحله به مرحله رو میتونید در تصویر بالا مشاهده کنید.

نکته مهم) صورت سوال خواسته تا واترمارک ۱۲ بیتی رو پیاده سازی کنیم پس ما تا بیت ۱۲م از
رشته ۲۴ بیتی داده شده پیش رفتیم ولی خب برای ۲۴ بیتی هم روال مشابه هست و نکته اضافی
ندارد.

((پایان))

